

Smart Bank Account Management System

Project Documentation

1 . Introduction

The Smart Bank Account Management System is a console-based Python application that simulates working of a real bank. The goal of this project is to help students understand how real software manage users, store data, validate operations, and handle errors.

This project does NOT require any graphical interface. The entire system runs in the terminal/console prompt using menus and user input.

By completing this project, students will learn how multiple Python concepts work together to build an application rather than isolated programs.

2 . Project Objective

The system should allow a user to:

- Create a bank account
- Login securely using account number and PIN
- Deposit money
- Withdraw money
- Transfer money to another account
- Check account balance
- View transaction history
- Save all data permanently in files

The application should behave like a simplified ATM machine.

3 . Learning Outcomes

After completing this project, students will be able to understand:

- How real applications store data permanently
 - How login authentication works
 - How to build menu-driven programs
 - How multiple functions interact
 - How to prevent invalid operations (like withdrawing more money than available)
 - How error handling improves software reliability
-

4 . Concepts Covered

This project covers the entire core Python syllabus:

Variables & Data Types – storing account information
Conditional Statements – validating login and transactions
Loops – menu repetition until user exits
Functions – separate operations like deposit/withdraw
Lists & Dictionaries – temporary data handling
File Handling – permanent database storage
Exception Handling – invalid input protection
Modular Programming – organizing code into logical parts

5 . System Modules

The project must be divided into logical modules.

Module 1 : Registration

The system collects new user details: • Account Number • Account Holder Name • PIN (password)
Balance

The information is saved permanently into a file. Each account must be unique.

Module 2 : Login Authentication

The user enters account number and PIN. The system verifies credentials from stored file records.

If correct → Allow access If incorrect → Show error message

This simulates ATM card verification.

Module 3 : Account Operations

After login, the user sees a menu repeatedly until logout:

- 1 . Deposit Money
- 2 . Withdraw Money
- 3 . Transfer Money
- 4 . Check Balance
- 5 . Transaction History
- 6 . Logout

Module 4 : Deposit

User enters an amount. The system adds the amount to the balance. Transaction must be recorded in history file.

Module 5 : Withdraw

User enters an amount. System checks: If balance is sufficient → Allow withdrawal If insufficient → error message

This teaches validation logic.

Module 6 : Transfer

User enters another account number and amount. The system must: • Verify receiver exists • Deduct from sender • Add to receiver • Record transaction for both users

This simulates real bank transactions.

Module 7 : Balance Inquiry

Displays current available balance of logged-in user.

Module 8 : Transaction History

Displays list of operations such as: Deposit Withdraw Transfer Sent Transfer Received

Each operation must include amount and operation type.

6 . Data Storage Design

The system uses text files as database.

Accounts File

Stores user account details permanently. Each line represents one user.

Data stored: Account Number | Name | PIN | Balance

Transactions File

Stores transaction history separately for each account. This allows tracking financial activity similar statements.

7 . Program Flow

- 1 . Program Starts
 - 2 . User chooses Register or Login
 - 3 . After login → Banking menu appears
 - 4 . User performs operations repeatedly
 - 5 . Data updates in file instantly
 - 6 . Logout returns to main menu
 - 7 . Exit closes application
-

8 . Validation Rules

The system must handle incorrect situations:

- Negative deposit not allowed
- Withdraw more than balance not allowed
- Transfer to invalid allowed
- Empty input not allowed
- Letters instead of numbers handled safely

This improves software reliability.

9 . Error Handling Requirements

The program must not crash when user enters: • Letters instead of numbers • Empty input • number

Instead it should display a meaningful message and continue running.

10 . Expected Output Behavior

The program should behave like an ATM machine:

- Always show menu after each operation
 - Confirm successful transactions
 - Prevent invalid operations
 - Store data permanently
 - Data should remain even after closing program
-

1 1 . Project Extension Ideas (Optional)

Advanced students may add:

- Password encryption
 - Admin panel
 - Interest calculation
 - Minimum balance rule
 - Account lockout on wrong attempts
 - Monthly statement generation
-

1 2 . Evaluation Criteria (For Students)

Students will be evaluated based on:

Program structure and organization
Correct working of login system
Correct handling of transactions
Proper file storage
Error handling
User friendly interaction

1 3 . Conclusion

This project demonstrates how basic Python concepts combine to form a real-world application. By completing this project, students will understand the logic behind ATM machines and banking software systems.

The main goal is not memorizing syntax but learning problem solving, validation, and program testing, which are essential skills for software development.